7. Pattern Recognition

This module is about how an animal recognizes a thing by the pattern it makes across many inputs, and recognizes it again when the pattern changes. While reading, look for the beginnings of answers to these questions:

- A fish has to treat two quite different smells as the same eel, and two similar smells as different things. How can both demands be met at once, and what decides where the line between them should go?
- Some problems cannot be solved by any single unit, however its weights are set. What makes a problem like that, and what has to be added to solve it?
- Psychologists have proposed that we sort things by comparing them with a summary of each category, or with every example we remember. Why has it been so hard to tell these accounts apart?
- A layer of units between input and output can recode a problem so that it becomes easy. What sets the weights of that layer?
- Current networks recognize images well, and still fail in ways no person would. Which of those failures are gaps in pattern recognition, and which are signs that something else is missing?
- The neocortex is built on one plan across its whole extent. What does that plan do with whatever reaches it?

By the end, we should have a working answer to the question Module 6 left open about the neocortex, and a clearer sense of what recognizing a pattern does and does not explain.

A small fish lives in a stream where eels hunt. Now and then, the water flowing past it carries the smell of an eel. But the fish never encounters the same eel smell twice. The mixture that makes up an eel's smell differs from eel to eel, and even for the same eel from one moment to the next. Each eel emits a complex constellation of chemicals that make up its smell. And those chemicals mix with everything else in the water: rotting leaves, other fish, the mud of the stream bed. Out of all that variation, the fish needs one question answered. Is a dangerous eel nearby, or not? And the answer is only useful if it comes while there is still time to get away.

Nothing in the fish's nostrils stands for an eel. Each kind of smell receptor responds to some chemicals, and not to others. And an eel gives off a mixture of many chemicals. So what identifies an eel is a pattern of activity across many receptors. And much of the same pattern turns up for fish that are no threat at all. A person who knows the smell of coffee is in the same position. There is no coffee molecule, and no coffee receptor. There is a pattern of chemicals that is characteristic of coffee.

The neural units we discussed in Modules 4 and 5 handled an easier case. In Module 4, a unit added up its weighted inputs, and fired when the sum crossed a threshold. As we described, this amounts to drawing one straight line or plane through the space of its inputs, and firing on one side of the line and not on the other.

In Module 5, we showed how experience and learning could move that line. Where one input, or one simple combination of inputs, reliably meant food or copper, one line was enough. But most of what matters to an animal has no detector of its own. Prey, predators, mates, and the way home all arrive as patterns across many inputs. And the patterns change from one encounter to the next.

At the end of Module 6, we left a question open. The pallium, the roof of the vertebrate forebrain, grew in mammals into the neocortex. It is a large, nearly uniform sheet. It has one kind of circuit across its whole extent, with the same six layers. This is true whether its signals come from the eyes, the skin, or the ears.

A structure that does the same thing to every kind of input raises a question. Is that one operation general enough to be useful to the eyes, the skin, and the ears alike? Or does the circuit add nothing of its own, so that what each area does, comes entirely from what is connected to it? The fish with which we began this chapter has no neocortex. But it has a pallium, and it faces the same problem any neocortex faces. It must recognize a thing from inputs that are never twice the same. So before asking what the neocortex computes, we need to be exact about what recognizing an eel requires (Table 7.1).

Table 7.1 — *Marr's three levels, the question each asks, the discipline that asks it, and this module's example.*

Level	The question it asks	Discipline	This module's example: recognizing a thing by its pattern
Computational	What problem is being solved, and why does solving it matter?	Evolution and ecology	Telling apart inputs that call for different responses, and treating alike inputs that come from the same thing, when no single input decides and no two encounters are the same
Algorithmic	What is represented, and by what procedure?	Cognitive psychology and AI	Networks that re-represent their input, using layers of units between the input and the decision
Implementational	By what physical means, and at what cost?	Neuroscience	The neocortex, and circuits in fish and insects that meet the same problem in other ways

§7.1 — Computational: The Pattern Recognition Problem

§7.1.1 — Recognizing by Pattern

An animal can recognize something in the world in two quite different ways. The first is the way the worm from Module 4 recognizes copper. A sensory neuron responds to copper, and its output is routed to the machinery that turns the worm away. Nothing else has to happen. The neuron firing *is*, in effect, the worm recognizing copper. In Module 4, we called an arrangement like this a *labeled line*, and noted that the label is not on the line itself. It is at the far end, in what the line connects to.

The second way is what we described happening in the small fish recognizing the smell of an eel. Nothing in its nostrils responds to eels and only to eels. An eel is identified by a combination of activity across many receptors, and no one of them identifies it alone. Identifying something from a combination of this kind is called **pattern recognition**. Most of what matters to a fish has to be recognized this way: its prey, its predators, a mate, and the way back to shelter.

Pattern recognition needs something between the receptors and the muscles. The fish has to end up in a state that *means* "eel". None of its receptors can supply that state by itself. So somewhere downstream of the receptors, activity has to arise that stands for the eel, whichever mixture of chemicals produced it. A pattern of activity that stands for something in the world, so that what the animal does next depends on what the pattern stands for, is called a **representation**.

In Module 5, we said that which pairings an animal learns easily is partly a question about representations. A new connection can only link things the nervous system already represents, so how easily two things are paired depends in part on how each was represented beforehand.

§7.1.2 — Telling Apart, Treating Alike

Recognizing an eel makes two demands at once, and the two pull against each other.

The first demand is **discrimination**. Inputs that call for *different* responses have to be distinguished, even when they overlap. Suppose a harmless fish, a sucker, shares the stream with the eel. And suppose its smell shares much of its chemistry with the eel's. The small fish has to flee from one, and ignore the other, although most of what reaches its nostrils is the same in both cases.

Discrimination usually improves with experience. Eleanor Gibson argued that learning to perceive does not consist of adding detail to thin, vague sensations. It consists of separating things that used to blur together. A novice wine drinker can tell red wine from white. An expert can also tell apart grapes and regions that taste the same to the novice. Experience has pulled apart inputs that the novice treats as alike.

The second demand is **generalization**. Inputs that call for the *same* response have to be treated alike, even if they barely overlap. A large eel upstream and a small eel downstream give off quite different mixtures, but both call for the same response: flight.

It is tempting to think of generalization as a kind of failure. The animal responds to a new eel as it did to an old one, because it cannot tell the two apart. The psychologist Roger Shepard argued the opposite. Generalizing is a bet that two things share a consequence. His example was an animal that eats a caterpillar and is made sick by it. On seeing a second caterpillar that looks a little different, the animal may tell the two apart perfectly well. What it cannot tell is whether the second one will make it sick too. Treating the two alike is a bet that it will.

A system that met only one of these two demands would fail. A fish that only discriminated would treat every eel as a new thing, and nothing it learned about one eel would carry over to the next. A fish that only generalized would lump the eel together with the sucker, and would flee from both or from neither.

The two demands (discrimination and generalization) are hard to satisfy at the same time, because they push on the same inputs in opposite directions. Getting better at telling the eel from the sucker means attending more closely to the ways their smells differ. But a large eel and a small eel differ in some of those same ways. So a fish that has sharpened its discrimination may begin to treat the two eels as different things. Going the other way, widening what counts as an eel so that every eel is caught, also widens it toward the sucker. The small fish needs both kinds of change for the same smells. It needs the smell of the eel pulled apart from the sucker, and also the smell of every eel drawn together into one kind of thing. It cannot get there by becoming more discriminating, or more general, across the board. Some differences have to come to count for more, and others for less.

The fish's input at any moment is a pattern of activity across its smell receptors. What the fish's world calls for is a mapping from inputs to responses: this pattern of receptor activity means flee, and that one means keep feeding. Discrimination and generalization are two demands on that single mapping.

§7.1.3 — How Alike Does Something Have to Be?

If generalizing is a bet that two things should be treated the same, how far from a known case should the generalization extend? The natural answer is that the more a new case resembles the known one, the more readily it should be treated the same way.

We can measure generalization without saying anything about what goes on inside the animal. Train an animal to give a response to one input, and then test it on inputs that differ from the trained one by different amounts. The tendency to respond to a new input as to the trained one falls off smoothly as the new input differs more, and often steeply. That fall-off is a **generalization gradient**. Researchers measured such gradients long before anyone spoke of representations. Pavlov found that a dog conditioned to salivate to one tone also salivated to other tones, and less the more they differed from the trained tone. Pigeons trained to peck at a light of one color also peck at lights of other colors, and less often the further the color is from the trained one. A gradient of this kind is a description of behavior: how strongly the trained response is given, plotted against how the test input differs from the trained one.

How the difference is measured matters, though. Shepard found that closeness measured in physical units is not a good predictor of how animals generalize. The same physical difference can matter a great deal on one dimension and hardly at all on another. And the pattern differs from species to species. A tone can be treated as more like the tone an octave above it, than like tones that are physically closer and right below. So the differences that govern generalization are not simply the physical ones. For the small fish, what matters is which differences between smells make a new smell more or less like an eel to it, and that has to be measured by watching what the fish does.

§7.1.4 — Deciding Under Noise

No two sniffs of an eel are the same, and the receptors and neurons that respond to them are noisy too. The evidence for "eel" that reaches the fish varies from one encounter to the next, even when an eel is there. The evidence when no eel is there varies as well. We can plot how often each amount of evidence

occurs, once for encounters with an eel and once for encounters without one, and show that the two distributions overlap. Somewhere along the evidence axis, a line has to be drawn: flee above it, keep feeding below it.

The framework for making decisions of this kind is called **signal detection theory**. It was developed in the 1950s for detecting faint signals on radar. It is now used wherever a decision is about whether something is there: a faint tone, a disease, a remembered face, a junk message. Any single decision has one of four outcomes (Table 7.2). When a target is present and judged to be present, this is called a **hit**. When a target is present and judged to be absent, it is a **miss**. When no target is present, but one is judged to be there, it is a **false alarm**. And when no target is present, and none is judged to be there, it is a **correct rejection**.

Table 7.2 — *The four outcomes of a detection decision, for the small fish deciding whether an eel is present.*

	The fish flees	**The fish stays**
An eel is present	Hit	Miss
No eel is present	False alarm	Correct rejection

Signal detection theory separates two quantities that are sometimes confused. **Sensitivity** is how far apart the two distributions lie. It is usually measured as d', read “d-prime”, the distance between their centers in units of their spread. The **criterion** is where the line is drawn. Moving the criterion trades one kind of error for the other. A lower criterion produces more hits and fewer misses, but also more false alarms. And moving the criterion leaves sensitivity exactly where it was, because the distributions have not moved at all (Figure 7.1).

Two people taking a face memory test show the difference. Each sees 50 faces they studied earlier and 50 new ones. One says “yes, I saw that one” to 42 of the studied faces and to 8 of the new ones, and is right 84% of the time. The other says “yes” to 49 studied faces and to 25 new ones, and is right only 74% of the time. But both have a d' of about 2.0. The second person simply says “yes” more often. Percent correct mixes sensitivity and criterion together, and signal detection theory pulls them apart. A smoke alarm that goes off whenever toast burns is often called sensitive. In these terms, it has a low criterion.

Which criterion is best depends on what each outcome gains or loses, and on how common targets are. When targets are rare and false alarms lose a great deal, the best criterion is conservative. When misses lose a great deal, it is liberal. So accuracy is the wrong measure when the two errors lose different amounts. In one worked example, a false alarm loses 15 points and a miss loses only 1. Accuracy is highest with a neutral criterion, but the points earned are highest with a conservative one. The poorer the sensitivity, the more extreme the best criterion becomes, which is why people walk more carefully in a dim room.

For the small fish, missing an eel may end its life, while a false alarm loses a few minutes of feeding. So the best criterion for this fish is liberal. A jumpy fish is wrong more often than a cautious one, but the cautious fish is eventually eaten. Whether any real fish sits near its best criterion can be answered only by measuring real fish. The two quantities are also different kinds of thing to change. The criterion can move with no change in how the fish represents what reaches its nostrils. Sensitivity can improve only through a better representation of the input.

§7.1.5 — Categories Have Structure

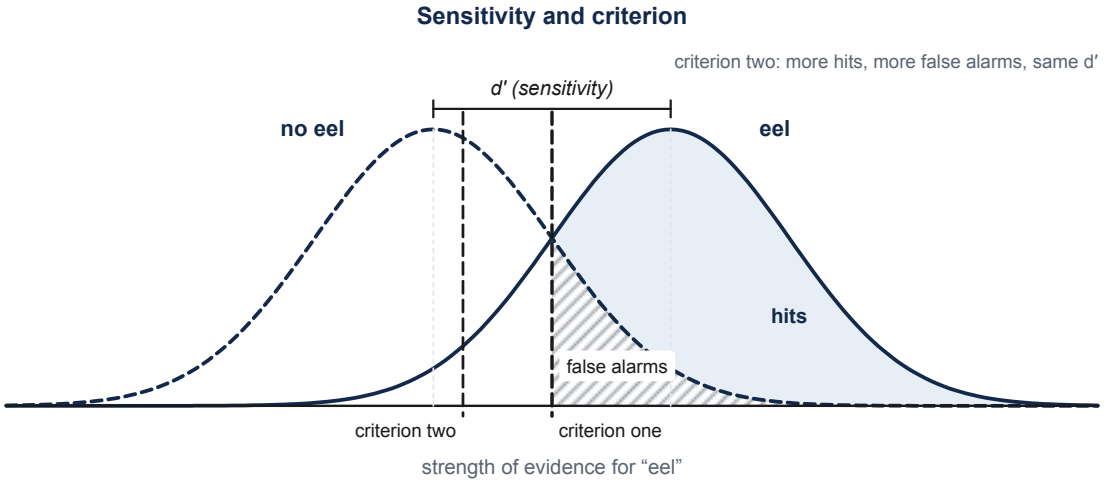


Figure 7.1 — *Two overlapping bell-shaped distributions of evidence, one for encounters with no eel present and one for encounters with an eel present. Two possible criteria are marked as vertical lines. For each criterion, the area of the eel distribution to its right is the hit rate, and the area of the no-eel distribution to its right is the false-alarm rate. Moving the criterion changes both rates but leaves the distance between the distributions, the sensitivity, unchanged.*

A **category** is a set of things treated as the same kind, for some purpose. And some members of a category are better examples of it than others. The psychologist Eleanor Rosch asked 209 students to rate members of ten everyday categories on a seven-point scale, from a very good example of their idea of the category to a very poor one. For birds, a robin was rated 1.02, close to the best possible score. A penguin was rated 4.53. A bat, which is not a bird at all, was rated 6.15, close to the worst. Different groups of students agreed closely on these ratings. This graded goodness of membership is called **typicality**.

A later study asked what makes one member more typical than another. People listed the attributes of twenty members of each of six categories, and few or no attributes were listed for all twenty. The members instead overlapped the way the faces in a family do. One member shares some features with a second, the second shares others with a third, and nothing runs through the whole set. This pattern is called *family resemblance*. The most typical members were the ones that shared the most features with other members of their own category, and the fewest with members of neighboring categories.

The evidence for typicality comes from human beings, and it is a fact that any account of categories has to explain. But it cannot be used to decide between accounts on its own, because several quite different accounts explain it equally well.

Not every category is held together by resemblance among its members. Some are built around a goal, such as things to take out of a burning house, whose members have almost nothing in common. Others are defined by a relation. Nothing about an animal’s own appearance makes it a predator. What makes it a predator is what it does to other animals. And sorting things is only one use for categories. Categories also let an animal infer properties it cannot see, and let people explain what happens and communicate. So recognition is not only a matter of how alike inputs are. It is a matter of which inputs share a consequence.

§7.1.6 — When Only Combinations Matter

For some categories, each input carries information on its own. A little more of one smell makes an eel a little more likely, whatever else is present. For other categories, no input carries any information on its own, and the right response depends only on the combination of inputs.

Consider a hypothetical animal that feeds on fallen fruit. Suppose that either of two smells alone means ripe fruit, while both together mean the fruit is rotting, and neither means there is no fruit at all (Table 7.3). Each smell, on its own, is present in half of the cases that mean food and in half of the cases that do not. So neither smell says anything about food by itself, and only the combination does. The rule is called *exclusive or*, often written XOR. This is the situation where one input is true, or the other, *but not both*.

Table 7.3 — *The four combinations of two smells for the hypothetical fruit-eating animal, and what each one means.*

	Smell B absent	**Smell B present**
Smell A absent	No fruit	Ripe fruit
Smell A present	Ripe fruit	Rotting fruit

A second case has the same character. Suppose a fruit smells right only when a chemical is at just the right strength, since too little means unripe and too much means rotten. More of the chemical is good news up to a point, and bad news after it. So the chemical's strength, on its own, does not point consistently toward either answer.

Animal-learning researchers discovered this problem before anyone was studying XOR in neural circuits, or before anyone built an artificial neural network. In some discrimination tasks, an animal is rewarded for going to the right when two cards are white, and for going left when they are black. Every single cue in that task is rewarded half the time. An account in which the animal learns only about single cues predicted that no animal could learn the task. Animals learned it anyway. A number of other studies since have shown that a range of animals, from rats to monkeys, are capable of learning tasks in which only combinations of cues predict the outcome.

People can learn categories that depend only on combinations, too. In controlled experiments, categories that could be sorted by adding up evidence one feature at a time were no easier to learn than categories that could not. And two practiced observers learned a category with the shape of exclusive or.

§7.1.7 — The Same Thing Experienced Differently

Sometimes, one thing can produce inputs that barely overlap. An eel close by, and an eel far away, produce smells at very different strengths. A face seen from the front shares little of its image with the same face turned to one side. Recognizing a thing across changes of this kind is called **invariance**. So the smells of one eel can differ from each other more than either one differs from the smell of a sucker. They still have to be treated as one thing.

This is discrimination and generalization at their hardest. In Module 1, we saw the reverse side of the same difficulty in vision. Many different scenes can cast exactly the same image. Invariance is the other side of that problem, since one object can cast many different images. For a nervous system, invariance means that some neuron downstream has to fire for every eel smell, faint or strong. And it has to stay quiet for the smell of the sucker, even when that smell is more similar to one of the eel smells than the eel smells are to each other.

§7.1.8 — Learning in Sequence

A fish does not learn what an eel smells like all at once. It learns across many encounters, and it learns about other things in between. Things learned at different times often overlap. The smell of a new predator may share much of its chemistry with the smell of the eel, and with the smells of harmless fish. So learning about one of them bears on what has already been learned about the others. New learning has to add to old learning, without overwriting it.

A concrete form of the problem appears whenever the same features turn up in objects that are rewarded, *and* in objects that are not. Then no single feature reliably predicts the outcome. Each one is followed by a reward on some occasions, and by nothing on others. Only combinations of features are

reliable. And the more objects share features, the more of the reliable information lies in combinations rather than in single features.

This is the trade-off we called stability versus plasticity in Module 5. It involves needing to change enough to learn something new, while changing little enough to keep what was learned before. For the small fish, learning the smell of a new predator must not erase what it knows about eels, even though the two smells are carried by many of the same receptors.

§7.2 — Algorithmic: Neural Networks and Pattern Recognition

§7.2.1 — Definitions and Rules

Some smells should send the fish fleeing and others should not. Knowing that is not the same thing as having a way to sort them. An algorithm is a procedure that does the sorting, using what the fish has learned from the smells it has already experienced.

The oldest answer to how an entity might, or should, form categories about the world is called the **classical view**. Under the classical view, a category is defined by features that are individually necessary and jointly sufficient to be a member of that category. Everything with all of the defining features belongs, and nothing without them does. For example, a *bachelor* is an “unmarried adult man”. An *odd number* is a “whole number that leaves a remainder of 1 when divided by 2”. According to the classical view, membership is all or none, and every member belongs equally. For a long time, the classical view was the default account of categories in both psychology and philosophy.

The classical view can be seen as a more complex version of the labeled line we discussed in §7.1.1. A labeled line is a single test. One kind of receptor responds, and a response follows. A definition is a set of such tests, combined by a rule. In the terms of Module 4 (which is our own translation rather than the classical theorists’), each defining feature could be detected by a labeled line, and the category by a unit computing AND over them, which fires only when every test is passed. Rules are not limited to AND. “Smell A or smell B, but not both” is a perfectly good rule, so a definition can capture the fruit of §7.1.6.

Definitions can also be stacked. A defining feature need not be detected directly, because it can have a definition of its own. A square is a closed figure with four straight sides of equal length and four right angles. None of those features is a single test either. Whether two lines form a right angle is itself defined in terms of simpler properties of the lines, and those in terms of simpler properties still. Built this way, a definition is a hierarchy of tests. At the bottom are tests simple enough for a receptor to perform, and each level up combines the results of the level below by a rule.

In Module 4’s terms, that is a hierarchy of units, each one computing a truth table over the units beneath it. But a hierarchy of this kind works only if every level can be defined in terms of the level below it, all the way down to tests that a receptor can perform. For a square, that can be done. For most everyday categories, such as *bird* or *chair*, it may be much harder to identify a chain of definitions that reaches all the way down.

The case against the classical view does not rest on what definitions can compute. It rests on what people’s categories turn out to be like. According to most cognitive scientists, the classical view is not the best explanation of how animals, especially humans, divide the world into categories. The typicality findings of §7.1.5 are the main evidence against the classical view as a general account. When people listed the attributes of twenty members of a category, few or no attributes were true of all twenty. So for most everyday categories, there is nothing for a definition to list. And a definition makes every member equally good, while people rated a robin a far better example of a bird than a penguin, and a bat worse still.

But typicality alone does not refute definitions. One study found that people rate some odd numbers, such as 3, as better examples of odd numbers than others, such as 447. The same people know perfectly well that the category has an exact definition, and that every odd number belongs to it equally. So graded ratings can exist alongside a definition. Some categories are probably learned as rules, and others are not.

In psychology and philosophy, the word *concept* is also used for something more than a category. A concept in that sense is a part of thought, which can be combined with other concepts to form new thoughts. Anyone who can think of a pet, and of a fish, can think of a pet fish. Whether definitions, or the

accounts that follow, can play that part has been argued for decades (or millennia), and we take the question up in Module 14.

§7.2.2 — A Space of Inputs

The most widely used alternative to definitions starts from a picture of the inputs. Give each kind of receptor an axis in a multidimensional sensation space. For example, the smell input at each moment is a point in that space, representing how strongly each smell receptor is responding.

We can simplify the example by imagining a “smell space” with only two axes, one for each of two kinds of receptor. In this scenario, one sniff of eel is a point on a flat 2D plane. Many sniffs of many eels make a scattered cloud of points. Sniffs of the harmless sucker fish make a second cloud, overlapping the first at its edge (Figure 7.2). A space of this kind, with one axis for each feature of the input and one point for each input, is a **feature space**. Inputs that are alike land close together in it, and inputs that are different land far apart.

A feature space gives us one way to create the map between inputs and outputs that the fish’s world calls for. The inputs that should get one response form a *region* of the space. The mapping becomes a division of the space into regions, one for each response, with a boundary between them. Where that boundary should run is set by the inputs themselves. It depends on which smells come from eels and which from suckers, and on how much the two kinds of smell overlap.

A real fish has many kinds of smell receptors, so its feature space has many axes. The geometry described above can be extended to higher dimensional spaces. With three axes, a straight boundary is a flat plane instead of a line. With more than three axes, we lose the ability to visualize the space. But the high-dimensional feature space can still be divided into two parts using a single hyperplane, the many-dimensional version of a flat plane.

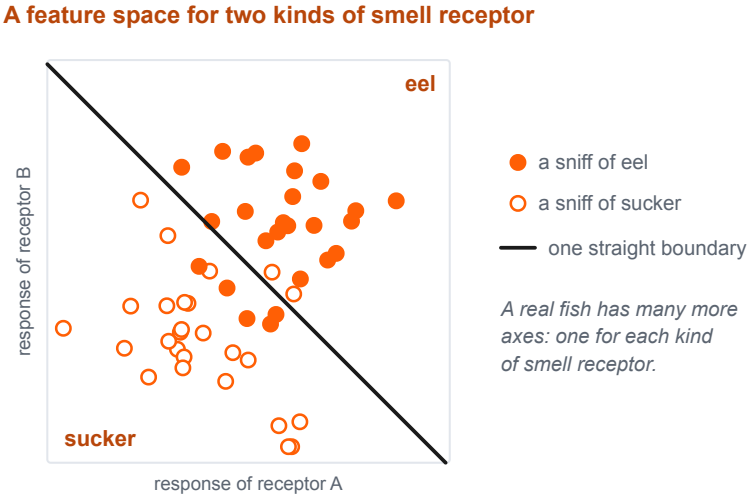


Figure 7.2 — *A feature space with two axes, each the response of one kind of smell receptor. Sniffs of eels form one scattered cloud of points, and sniffs of a harmless sucker form a second cloud that overlaps the first at its edge. A single straight line divides the space, with most eel points on one side and most sucker points on the other, and a few points of each on the wrong side.*

Some divisions of a feature space are simpler than others. Two categories are linearly separable when a single straight line, or a single flat surface, can be drawn through the feature space with all of one category on one side, and all of the other category on the other side. That property is **linear separability**. The eel and sucker clouds in (Figure 7.2) would be linearly separable if they did not overlap, since the diagonal line would then put every eel on one side and every sucker on the other. The

AND and OR of Module 4 are linearly separable too, which is why a single unit could compute each of them.

But there are many relationships in the world where no single line, plane, or hyperplane can divide the space correctly into two responses. The fruit we described in §7.1.6, where either smell alone means ripe fruit and both together mean rot, is just such a case. Draw the four cases in a feature space, with one axis for each smell. The two cases where food is present are at opposite corners. And so are the two cases where no food is present. No straight line can put both food corners on one side, and both other corners on the other (Figure 7.3). The fruit whose chemical has to be at just the right strength is the same. The acceptable smells are an island in the middle of the space, with unacceptable ones on every side. A straight line can cut off one side of the island, but it cannot wrap around it.

Whether one straight line is enough depends on which features the space is built from. Add a third input that is active only when both smells are present, the product of the two, and the rotting corner moves away from the other three. Now a single flat plane separates the food cases from the rest. The island works the same way. Add an input that reports how far the chemical is from its ideal strength, its squared distance from that point, and a single threshold on that one input marks off the island. With the right features, the hard problems of §7.1.6 become easy. The question is where such features come from. They can be learned, wired at random, or built in, and each of those answers comes up later in this section.

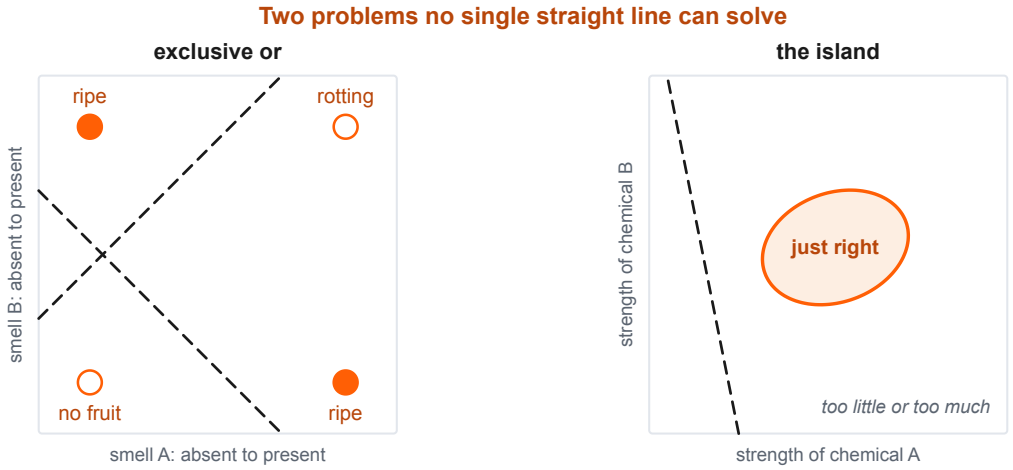


Figure 7.3 — *Two feature spaces that no single straight line can divide. On the left, the exclusive-or pattern: the four combinations of two smells, with the two food cases at opposite corners and two attempted straight lines, each of which leaves one case on the wrong side. On the right, an island of acceptable smells surrounded on all sides by unacceptable ones, with a straight line that cuts off only one side of it.*

Psychologists have proposed three broad families of algorithm that treat an input as a point in a feature space. They differ in what has to be stored, and consulted, before a response can be made. And every one of them ends up drawing a boundary between categories, whether it stores that boundary or not. Those boundaries can be used to compare the families, since two models that draw the same boundary are very hard to tell apart in data.

§7.2.3 — Drawing the Boundary Directly

The most direct family learns the boundary itself. A **decision-bound model** divides the feature space into regions, one for each response, and responds according to the region an input falls in. Decision-bound theory grew out of signal detection theory. The criterion of §7.1.4 is a boundary on a single axis, and a decision bound is the same idea in a space with many axes. Some decision bounds have a simple

verbal form, such as “large squares belong to category A”, which is a rule. Many do not. In a feature space, a classical definition is a decision bound of this kind, built from straight cuts along the feature axes, one cut for each defining feature, with the category in the region where every condition is met.

The simplest decision-bound model is a single unit of the kind in Module 4. Its boundary is a straight line, its **decision boundary**, with firing on one side and silence on the other. Changing the unit’s weights turns the line, and changing its threshold or baseline firing rate slides it. That is how one unit computed both AND and OR in Module 4: their two lines are parallel, and going from one to the other only slides the line. A unit’s weighted sum is its evidence, in the sense of §7.1.4, and its threshold is its criterion. Every combination of inputs that gives the same sum lands on the same point of the evidence axis, so the decision boundary is exactly the set of inputs whose evidence sits at the criterion.

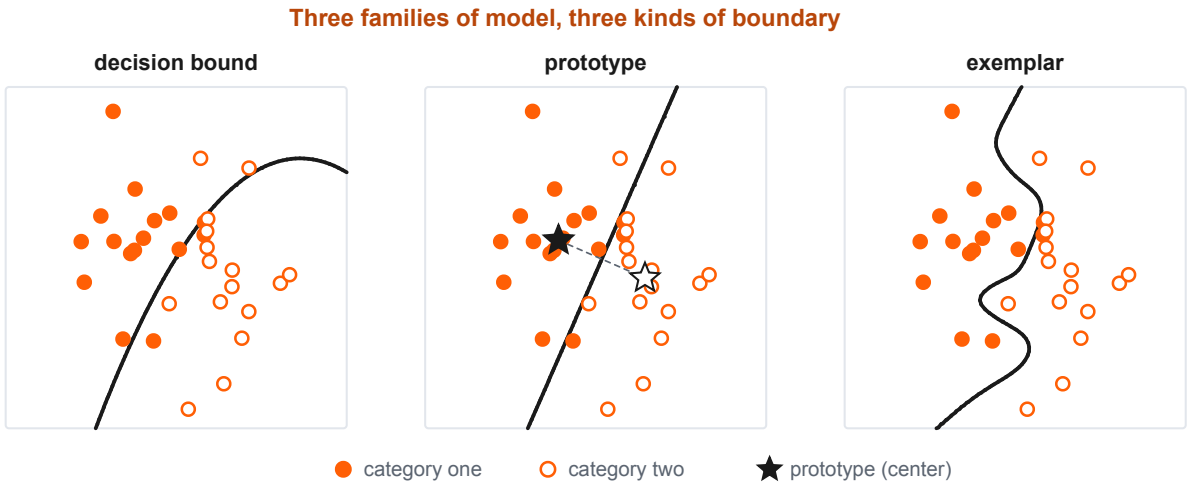


Figure 7.5 — *Three families of categorization model, and the boundary each one draws between two overlapping categories in the same feature space. Left: a decision-bound model stores the boundary itself, here a curve, and responds according to the side an input falls on. Middle: a prototype model stores one center for each category, and its boundary is the straight line halfway between the two centers. Right: an exemplar model stores every member, and its boundary is wherever summed similarity to the two categories is equal, which curves around the stored examples.*

In Module 4, we said that there are computations no single decision boundary can produce, however the weights and the threshold are set. Exclusive or, from §7.1.6, is the simplest of them. An animal or a person that learns exclusive or has something inside it more complex than a single neural circuit drawing a single line. Decision-bound models in psychology are not limited to straight lines, though. A curved boundary can wrap around the island of §7.1.6.

§7.2.4 — Prototypes, Exemplars, and Nearest Neighbors

The other two families do not store a boundary at all. They store something about the members of each category, and the boundary follows from what they store. They then make their decisions in terms of some measure of **similarity**, or closeness in feature space. Two inputs are similar when their points lie near each other in the space. But this similarity can be estimated in a number of different ways.

A **prototype** model stores one summary of each category, its central tendency, something like the average member. A new input is compared with each category’s prototype and assigned to whichever prototype it is closest to. An **exemplar** model stores the members themselves. A new input is compared with every stored member of every category. Its similarity to each one is added up, category by category, and the input goes to the category with the largest total. Each stored example contributes according to how close it is, so the generalization gradient of §7.1.3 does the work. A nearby example counts for a great deal, and a distant one for almost nothing (Figure 7.5).

Both prototype models and exemplar models rely on similarity. But they both raise questions about what space, exactly, is used to compute that similarity? As we saw in §7.1.3, when similarity is measured in physical units, like raw sensory inputs, similarity tends to not be a good predictor of human or animal behavior. Instead, a better predictor is the distance in what is called a *psychological space*, which we can recover from the behavior itself. We can measure the extent to which an animal treats each pair of inputs as the same. We can then create a similarity space that shows how the animal treats the inputs.

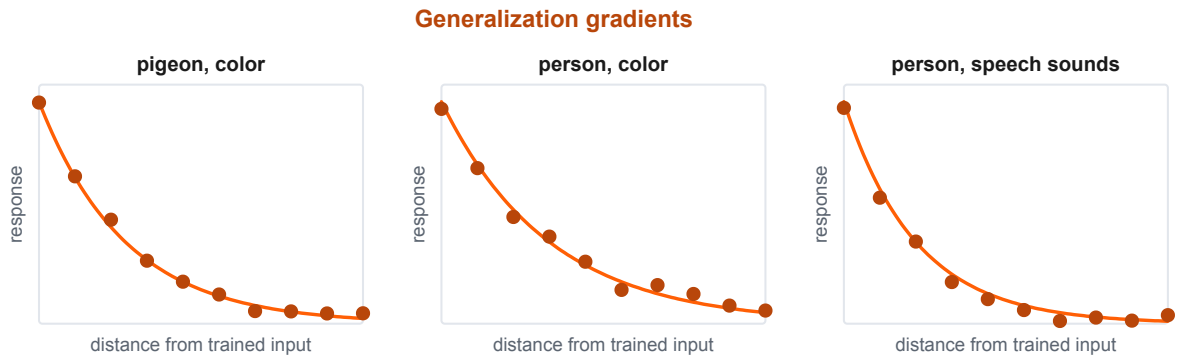


Figure 7.4 — *Generalization gradients. In each panel, the horizontal axis is distance from a trained input in a space recovered from behavior, and the vertical axis is how strongly the trained response is given to a new input. The curves fall smoothly and steeply as distance grows, in pigeons and in people alike. Schematic, drawn after the shape of the gradients Shepard (1987) reported; the points are illustrative, not his data.*

In a space created that way, generalization gradients are remarkably regular. Responding falls off smoothly, and steeply, as a new input gets further from a trained one (Figure 7.4). One review gathered twelve such gradients, from people and from pigeons, for inputs ranging from colors to speech sounds. They all fell off in nearly the same way, close to exponentially. The claim was limited to learning from a single experience tested right away, and with long training on similar inputs the gradient changes shape. So it is a strong regularity, not a law without exceptions.

Which dimensions count can itself be learned. When a task depends on one dimension more than another, attention can stretch the space along the dimension that matters, so that small differences along it become large distances. This is the expert wine drinker of §7.1.2, described as geometry. The dimensions that separate one grape from another have been stretched, and wines that once sat close together now sit far apart. For a nervous system, a stretched dimension would be one that neurons downstream respond to strongly, so that a small difference along it produces a large difference in their activity. The small fish would need the dimensions that separate the eel from the sucker stretched in just this way.

A simple relative of the exemplar model consults only the closest few examples. In **k-nearest-neighbor classification**, a new input is compared with every stored example. The *k* most similar are found, and the input is assigned to whichever category is most common among those neighbors. Here *k* is a number chosen in advance, such as 1, 5, or 15. It is one of the oldest methods in machine learning, and it is still widely used. Where an exemplar model lets every stored example vote, weighted by its similarity, a *k*-nearest-neighbor classifier gives the *k* closest examples one equal vote each, and the rest no vote at all. With enough stored examples, its boundary can follow almost any shape.

In a feature space, a typical member of a category is close to many others, and an atypical one is out at the edge. Prototype and exemplar models both explain typicality, because a typical member is close to the prototype and also to many stored examples. So typicality alone cannot tell the two apart.

One classic result looked at first like strong support for prototypes. In one study, people were taught to categorize distorted versions of two dot patterns. Later, people classified the undistorted originals nearly as well as the distortions they had studied, though they had never seen the originals. But an exemplar model predicts the same result, because each original sits in the middle of its distortions and is

similar to many of them at once. Later studies built categories in which closeness to the prototype, and closeness to particular examples, pointed in different directions. For about twenty years, experiments tested human and animal learning on comparisons of this kind. These experiments have mostly, but not always, favored exemplar models.

§7.2.5 — The Limits of a Single Prototype

The families also differ in what kinds of categories they can learn at all. Their boundaries show why. A prototype model that assigns each input to the nearest prototype draws a straight boundary, halfway between the two prototypes, and at right angles to the line joining them. Weighting some features more than others tilts that line, but leaves it straight. A prototype model of this kind turns out to be very hard to tell apart from a decision-bound model that draws a straight line. A model with one prototype per category shares the limitation of a single unit: it can only sort categories that are linearly separable.

Exclusive or makes the limit vivid. Return to the fruit of §7.1.6. The two cases that mean ripe fruit sit at opposite corners of the feature space, so their average is the center of the square. The two cases that do not mean ripe fruit sit at the other two corners, and their average is the same point. Two categories with the same prototype cannot be told apart by their prototypes, however the comparison is made.

An exemplar model has no such limit, because it keeps the cases. A new input that matches the “smell A only” example is close to a ripe case, whatever the average says. An exemplar model’s boundary can curve, and follow the stored cases wherever they lie.

In principle, then, the three families are different algorithms, and they draw boundaries of different shapes. The simplest prototype model draws a straight line, an exemplar model a curve that follows the stored examples, and a decision-bound model whatever boundary it has learned. That makes them look easy to compare.

In practice, distinguishing the models has been hard. Each family contains many specific models. Some prototype models keep several prototypes for each category, adding a new one when an item does not fit the old ones. Models like these are not limited to straight lines. It has been argued that for any prototype model, an exemplar model could be built that behaves the same way, and the reverse. A related point is that two models that draw the same boundary will be hard to tell apart from behavior, even when they work very differently inside. Decades of experiments have not produced agreement about which family best describes people. Many researchers now think people use more than one.

In this chapter, a *network* means an artificial neural network: a model built from units and weighted connections of the kind introduced in Module 4, run on a computer. A network describes an algorithm. Whether brains carry out the same algorithm is the question of §7.3. Put in network terms, which is our own translation rather than the psychologists’, an exemplar model behaves like a layer of units sitting between the input and the decision. There is one unit for each stored example, and each unit responds to inputs close to its own example. That layer is what lets an exemplar model handle exclusive or.

§7.2.6 — A Layer in Between Input and Output

A **hidden layer** is a layer of units between the input and the output. Its units are not driven directly by the world, and their activity does not directly drive behavior. What they stand for depends on their weights. The hidden units come to stand for combinations of the inputs, and a network can use them to solve problems a single unit cannot.

Here is one network that computes exclusive or for the fruit (Figure 7.6). It has two hidden units. The first fires when either smell is present. It computes OR. The second fires unless both smells are present. It computes NOT-AND, so it is silent exactly when the fruit is rotting. The output unit fires only when both hidden units fire. It computes AND. Work through the four cases, and the output fires for either smell alone, and for neither of the other two cases. Each of the three units computes a function a single unit can manage. None of them could compute exclusive or alone. Together they do.

The same network can be read in three ways, and each uses tools from earlier modules. Read logically, each unit computes a truth table of the kind we built in Module 4, and the network composes truth tables. Using AND to combine OR and NOT-AND, results in the computation of exclusive or. Read as boundaries, each hidden unit draws its own straight line through the feature space, and the output combines the two lines into a band between them. The band holds the two ripe corners and excludes the other two, which no single line could do. Read geometrically, the hidden layer remaps the inputs from their original positions in the input space, to new positions in the space of hidden-unit activity. In that

space, the two ripe cases land on the same point, and the other two cases land elsewhere. A single line now separates them.

The three interpretations describe one fact. The hidden layer re-represents the input, so that the last step is easy. At the level of the animal, the second hidden unit is a neuron whose activity means “not rotting”. No smell receptor carries that information, and the animal needs it.

If a hidden layer solves the problem, why was exclusive or ever a problem? Originally, it was because someone had to set those weights by hand. Frank Rosenblatt’s perceptron of 1958 already had a middle layer between its input and its output. That layer was wired at random and never trained, and learning happened only at the output.

In 1969, Marvin Minsky and Seymour Papert published a careful analysis of what networks without trainable hidden units cannot do. It was widely understood that a hidden layer could, in principle, recode a problem so that the output could solve it. What no one had figured out was a general way to train the hidden layer, so that training, rather than a person, produced the right recoding. Output units can be trained using an algorithm like the delta rule. Compare the unit’s output to the actual correct answer, and adjust weights appropriately to make the unit more likely to output the correct answer. But how can the hidden units’ weights be adjusted this way, when no one knows what each hidden unit’s correct output should be?

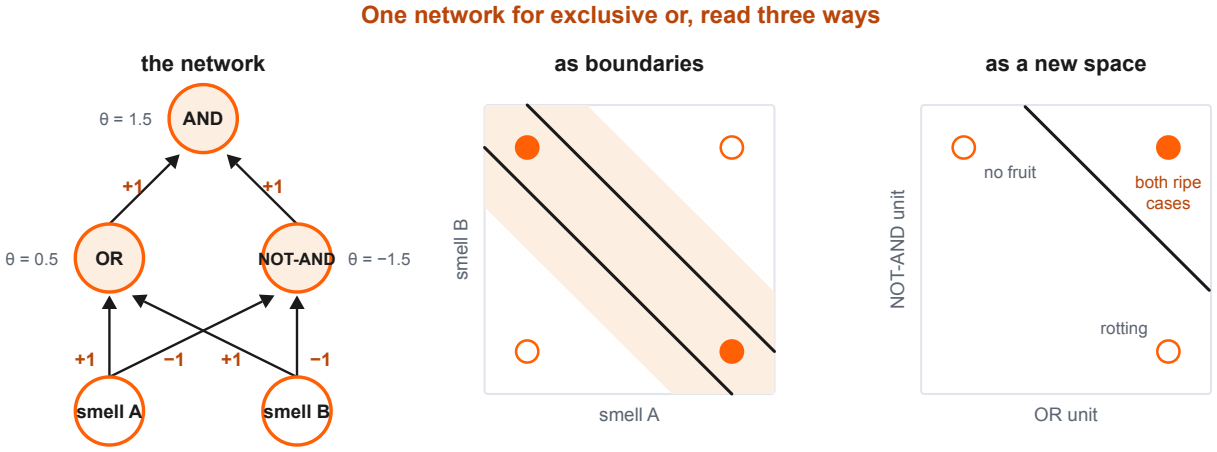


Figure 7.6 — *One network that computes exclusive or, read three ways. Left: the network, with two inputs, two hidden units computing OR and NOT-AND, and an output computing AND, with the weight on each connection marked. Middle: the feature space of the two smells, with the two hidden units’ lines drawn and the band between them shaded, holding the two ripe cases. Right: the space of hidden-unit activity, where the two ripe cases land on one point and a single line separates them from the rest.*

§7.2.7 — Three Ways to Set the Hidden Weights

Training a hidden layer runs into a problem of credit. In Module 5, credit assignment was the problem of working out which of several actions, or which of several connections, was responsible for an outcome that arrived later. A network with a hidden layer faces a version of this problem, in space rather than time. When the output is wrong, which connections deep inside the network were responsible? That is called **structural credit assignment**. There are three broad answers (Table 7.4).

The first answer is to pass the error back from the output to the hidden units. **Backpropagation** extends the delta rule of Module 5. Each connection still changes in proportion to the error at the receiving unit, times the activity of the sending unit. A hidden unit’s error is computed from the errors of the units it feeds, passed backward through the same connections the activity passed forward through (Figure 7.7). Each unit’s share of the error is also scaled by how much its output would change if its input

changed. Backpropagation is Module 5's gradient descent, run through every layer at once. It was made widely known in 1986.

The strength of backpropagation is that training finds the combinations the task needs. Its cost is that every output needs a target. But the target need not come from a teacher. In the prediction learning of Module 5, the target is the next input. Jeffrey Elman trained networks on nothing but a stream of words, with the task of predicting the next word. Their hidden layers came to group the words into nouns and verbs, and the nouns into animals, people, and inanimate objects, although nothing in the input said which words were which. Whether a brain could pass errors back in this way is a question for §7.3.

The second answer is that *nothing* sets the hidden weights. The original perceptron was built this way. A large middle layer is wired at random and never trained, and only the connections to the output are adjusted. With many more middle units than inputs, each combining a random handful of inputs, some unit is likely to respond to "smell A without smell B" without anyone arranging it. Spreading an input across many more units is **dimensionality expansion**. A straight boundary can separate far more sets of points in a space with many dimensions than in a space with few. The expansion works best with **sparse coding**, introduced in Module 3, in which only a few units are active for any one input. The cost is this procedure takes a great many units, and has no guarantee that the combinations a task needs will be among them. The fruit fly's brain has an arrangement of this kind, which we look at in §7.3.

The third answer needs no teacher at all. In **competitive learning**, units that receive the same input inhibit one another until one wins, like the winner-take-all circuits of Module 4. The winner moves its weights a little toward the input. That is a Hebbian step, kept in check because only the winner learns. Over many inputs, each unit comes to sit at the center of a cluster of similar inputs. If the winner's neighbors also move a little, neighboring units come to respond to similar inputs. The result is a **topographic map**, the arrangement of Module 6, arising from experience alone. The limit is that competition finds the structure in how inputs overlap, which is not always the structure the animal needs.

Table 7.4 — *Three answers to who sets a hidden layer's weights.*

Answer	What the weight changes depend on	Which weights change	What the hidden layer comes to represent	A named case
Error-driven learning	An error measured against a target, supplied by a teacher or by the world's next input	Every weight in the network	Whatever combinations reduce the error on the task	The digit-reading network (§7.2.9); Elman's next-word networks
Fixed random wiring, trained output	For the output connections only, an error or a reward	Only the connections from the hidden layer to the output	Whatever combinations the random wiring happened to produce	The perceptron; the mushroom body of insects, as proposed
Competition	Nothing outside the network; only the inputs and the competition among units	Only the winning unit's weights	Clusters of inputs that overlap, whether or not they matter for the task	Topographic maps (§7.2.7); the neocognitron (§7.2.9)

§7.2.8 — One Unit per Thing, or Patterns Across Many

What does a single hidden unit stand for? There are two broad possibilities. In a **localist representation**, each thing has its own unit, so that which unit is active is itself the information. In Module 3, we noted that every spike is the same size, so that part of a spike's meaning lies in which cell fired it. A localist code is that idea taken all the way. In a **distributed representation**, each thing is a pattern of activity across many units, and each unit takes part in representing many things.

Each kind of code has its own strengths. Distributed codes make similar things look similar without any extra work, because similar things share active units. They degrade gracefully when units are lost, and they pack many patterns into few units. Localist codes are easy to read and keep things cleanly apart. Which kind brains use is still argued, and a mixture of the two is a serious option.

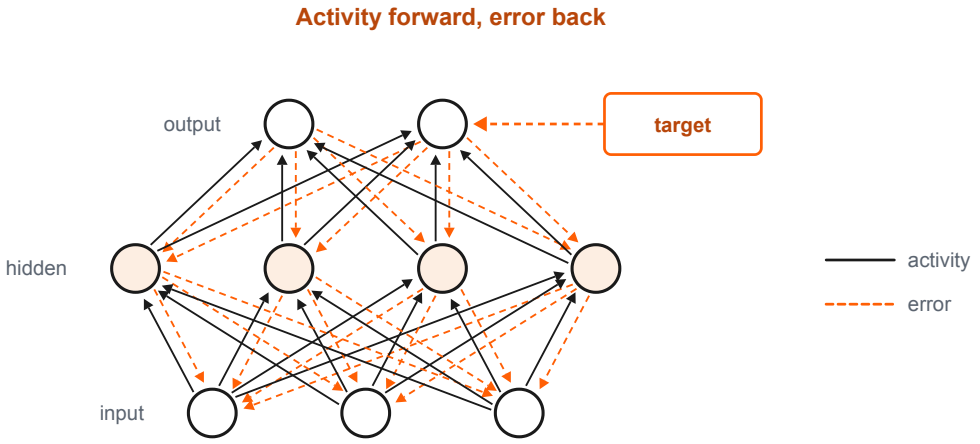


Figure 7.7 — *A network with an input layer, a hidden layer, and an output layer. Arrows of one color run upward, carrying activity from the inputs through the hidden units to the outputs. Arrows of a second color run downward along the same connections, carrying error from the outputs back to the hidden units.*

§7.2.9 — Stacking Layers for Invariance

Invariance was the hardest demand of §7.1.7. A network can meet part of it through its architecture, by building in assumptions about which changes do not matter.

Convolution applies the same small feature detector at every position in the input. A detector for a short vertical edge, for example, is copied across the whole image, with the same weights in every copy. So the feature is found wherever it falls, and when the input shifts, the map of where the feature was found shifts with it. Convolution alone does not make the response unchanged by a shift. It makes the response move along with the input. **Pooling** then combines the detections from a small neighborhood, so that a small shift no longer changes the pooled output (Figure 7.8). Stack several rounds of convolution and pooling, and each layer responds to combinations of the features in the layer below, over a wider region of the input. Small shifts and distortions of the input are tolerated a little more at each layer.

One architecture of this kind was built twice, with two different answers to who sets the weights. Kunihiko Fukushima's *neocognitron*, in 1980, had local detectors, shared weights across positions, and pooling, and its weights were set by competition, with no teacher. Yann LeCun and his colleagues, in 1989, kept the local, shared-weight design, but trained every weight by backpropagation. Their network read handwritten digits taken from zip codes on mail, and had to say which of the ten digits each one was. This digit-reading network learned from several thousand examples whose correct labels were supplied, and was then tested on examples it had never seen.

Building an assumption into an architecture is a bet about the task. The bet in a *convolutional neural network* is that a feature means the same thing, wherever it appears. That is an **inductive bias**, a term we introduced in Module 5. When the bet is right, it helps the network learn. Here, we can see an inductive bias built into the wiring of a network, rather than into the lineage of an animal. The digit-reading network had 64,660 connections but only 9,760 adjustable weights, because so many were shared. On digits it had not seen, it made 5.0% errors, against 8.1% for a fully connected network with about as many weights.

Convolution and pooling build in tolerance to one kind of change, a shift in position, and that tolerance is imperfect. But they build in nothing for other changes, such as size, rotation, or viewpoint.

For the small fish, the eel to the left or to the right is a shift. But an eel near and far is a change in size, and an eel from the front and from the side are different viewpoints. Artificial neural networks that recognize objects across changes of size and angle mostly get there by training on examples that already include those changes, often made by copying each training image shifted, rotated, rescaled, or blurred. Whether training of this kind is enough to explain how animals recognize things across such changes, is a live dispute.

§7.2.10 — What Shared Weights Give, and Take Away

When the patterns a learner stores are carried by the same units, learning a new pattern moves weights that an old pattern depends on. So new learning can disturb what was learned before. The feature-ambiguity problem of §7.1.8 shows the simplest case. A learner that sees only single features raises a feature's weight each time the feature is rewarded, and lowers it each time the feature is not rewarded. When a feature appears on two kinds of objects, the learner keeps undoing its own learning. And the more objects share features, the worse this gets. Representing the combination of features stops the interference, because a combination can be reliably rewarded even when none of its parts is.

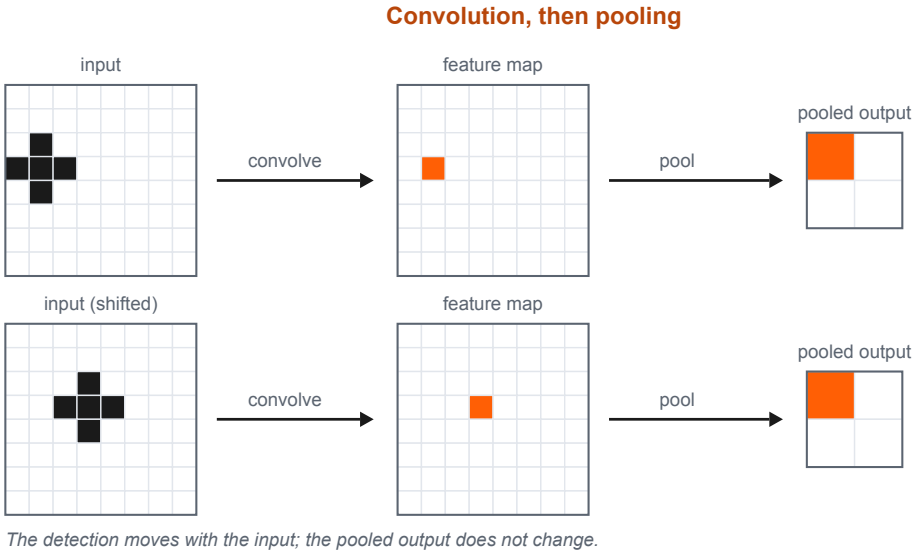


Figure 7.8 — *Convolution, then pooling. A small pattern appears in an input grid, and again in a second grid shifted a few positions to the right. Convolution with a detector for that pattern produces a feature map in which the detection appears at the corresponding position, so the feature map shifts with the input. Pooling over neighborhoods of the feature map produces an output that is the same for both inputs.*

In a network trained by backpropagation, the disturbance can be extreme. In a classic demonstration, a network was trained on the addition facts about "ones", such as 1 + 4 and 6 + 1, until it knew all of them. Then it was trained on the facts about "twos". After a single pass through the twos, the network began answering ones problems as if they were twos problems: 5 + 1 came out as 7, and 6 + 1 as 8. Its score on the ones fell from 100 percent to 57 percent, by the most lenient criterion, and to 30 percent after a second pass. When new learning wipes out old learning this drastically, it is called **catastrophic interference**.

The cause is the same property that makes the network useful. Every weight takes part in many different input–output pairs. That sharing is why similar inputs get similar outputs without any extra work, which is generalization. It is also why changing the weights for new pairs changes the answers to old ones. Generalization and interference are the same effect, seen from two sides.

The trigger is learning in sequence. Trained on all the facts at once, the same network learns them all (Figure 7.9). Trained on one set and then another, it moves its weights to wherever suits the newest set,

regardless of where the old set needed them to be.

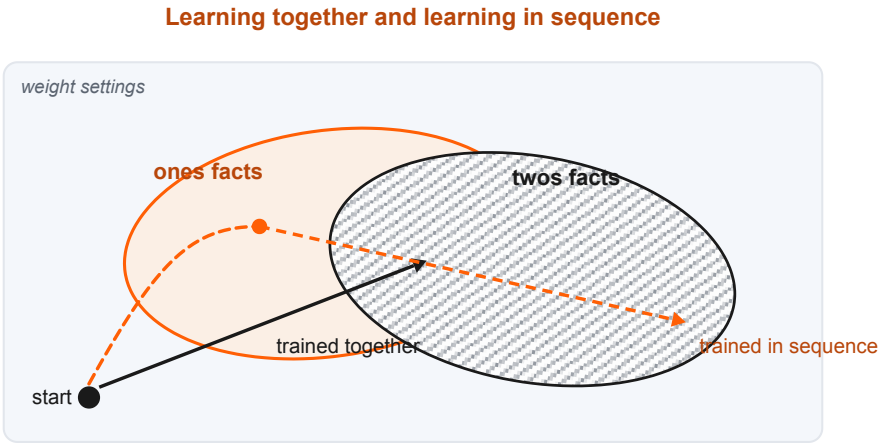


Figure 7.9 — *Learning two sets of facts, pictured in a space of weight settings. Each set of facts is learned by any weight setting inside its own shaded region, and the two regions overlap. Training on both sets at once moves the weights into the overlap, where both are known. Training on one set and then the other moves the weights into the first region, and then out of it to the nearest point of the second, so that the first set is lost.*

People forget too. When people learn a new list of associations that conflicts with an old one, they lose some of the old list. The difference lies in how much they lose. In a classic study, people still recalled more than half of an old list after mastering a new one, while the network lost all of its old list by the time it had learned about a fifth of the new one. In artificial networks, engineers have found ways to slow the loss, such as protecting the weights that matter most for old learning, or mixing old examples back into new training. How brains keep old learning while adding new, we take up in Module 10.

§7.2.11 — Fooled by Small Changes

In Module 1, we listed the characteristic ways image-recognizing networks fail. They can be fooled by changes a person would not notice. They lean on surface texture over shape. And they generalize poorly when conditions shift. We can now see some of the reasons.

An **adversarial example** is an input changed slightly and deliberately, so that a network gives a confident wrong answer. In one well-known case, a network labeled a photograph of a panda “panda”, with 57.7 percent confidence. After a carefully chosen pattern was added, the network labeled the same photograph “gibbon”, with 99.3 percent confidence. The change at each pixel can be about as small as an image file can record, and can be imperceptible to a human observer, while still changing the artificial neural network’s behavior.

One explanation makes the effect easy to understand with a tool from Module 4. A unit computes a weighted sum over its inputs. An image has many thousands of inputs. Move every input a tiny amount, each in the direction its weight favors, and the tiny changes add up. No single pixel moves much, but the total crosses the decision boundary (Figure 7.10). On this explanation, even a single unit should be fooled, and it is. A single unit that told handwritten 3s from 7s with 1.6 percent error misclassified 99 percent of inputs nudged in the direction of its own weights. The direction of the change is what matters. Random noise of the same size rarely changes the answer. The researchers who first reported adversarial examples proposed a different cause, and the explanation is still debated.

A second divergence is **texture bias**. Given a picture with the shape of a cat and the skin of an elephant, people say “cat”. Networks trained on a standard collection of photographs mostly said “elephant”. When the choice was between shape and texture, people chose shape about 96 percent of the time, and the networks between 17 and 43 percent. The bias came from the training images rather than from the architecture. The same architecture, trained on photographs whose textures had been

scrambled by repainting them in the styles of different paintings, chose shape 81 percent of the time instead of 22. That is clean evidence that what a classifier learns depends on its experience, and not only on how it is built.

§7.2.12 — Near the Examples, and Far from Them

A network's score on the examples it was trained on is not the measure that matters. A network can fit its training examples closely, including the accidents in them, and still do poorly on new examples. That is **overfitting**. Because of this, artificial classifiers learn from a **training set**, and then are evaluated on a separate **test set**, examples kept apart and never used in training. The digit-reading network of §7.2.9 made 0.14 percent errors on its 7,291 training digits, and 5.0 percent on 2,007 new ones.

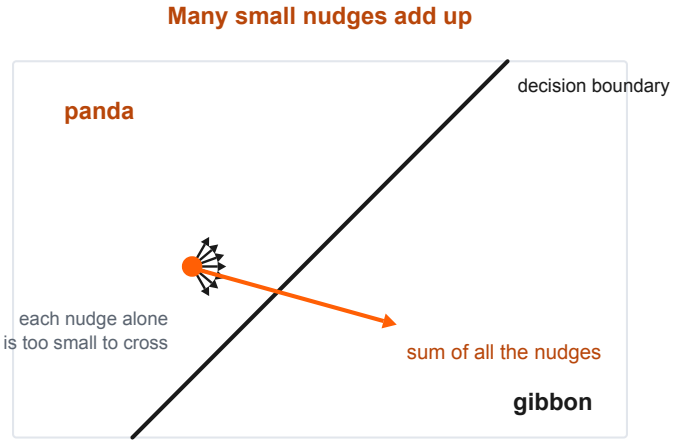


Figure 7.10 — *A decision boundary in a space with many dimensions, drawn as a tilted line. An input point sits a short distance from the boundary on one side. A small arrow moves the point a tiny amount along every axis at once, each in the direction the boundary favors, and the combined movement carries it across the boundary.*

Even a good score on a test set is a score on inputs like the training set, since the test set is drawn from the same collection of examples. **Out-of-distribution generalization** is generalization to inputs unlike anything in training. Current classifiers are much weaker at generalizing to out-of-distribution items. They interpolate well among familiar examples, and extrapolate poorly beyond them.

Arithmetic in early large language models is a good demonstration of bad out-of-distribution performance. These networks were trained only to predict the next word in a huge collection of text, and were never taught arithmetic. Yet they answered two-digit addition problems perfectly, and most three-digit problems correctly, though almost none of the three-digit problems appeared anywhere in their training text. Numbers of that size are common in text, so a new problem sat among many familiar ones. With four-digit numbers they were right about a quarter of the time, and with five-digit numbers about one time in ten.

In the terms of §7.2.2, the training examples occupy a region of the feature space. A classifier is reliable near that region, and much less reliable far from it. The same holds for invariance. A network trained on objects seen from familiar angles often fails on the same objects seen from unusual ones.

§7.2.13 — Gaps in Recognition, or Something Else Missing?

Not every failure is the same kind of failure (Table 7.5). Some are gaps in pattern recognition itself: adversarial examples, texture bias, overfitting, and failure out of distribution. A better classifier could, in principle, close them. Catastrophic interference is a gap of a different kind. It lies in the learning procedure, and it appears when learning happens in sequence.

Others look like marks of capacities that pattern recognition alone does not supply. Recognizing a refrigerator is pattern recognition. Knowing that it is heavy and rigid, and what would happen if it were pushed, may draw on something more. So may reasoning about causes. So may **systematicity**, the ability to combine known parts in new ways, so that anyone who understands “the eel chased the fish” also understands “the fish chased the eel”. We take up causes and planning in Module 12, language in Module 13, and symbols in Module 14.

A similar argument has been made about the psychological models of §7.2.4. Prototype, exemplar, and nearest-neighbor models all rest on similarity. Some researchers argue that similarity cannot be what holds a category together. Anything is similar to anything else in some respect. A cloud and a white feather are very similar if we compare their color and their ability to float in the air. They are very different if we compare their size and what they can be used for. So similarity alone cannot say which respects matter.

On one alternative, a category is held together by a theory, an understanding of why its members have the features they do. In one study, children were told about a raccoon that doctors had changed to look and smell like a skunk. Older children usually said the animal was still a raccoon. Their answer followed what the animal was underneath, not what it looked like. Knowing what an animal is underneath goes beyond its pattern, in the same way that knowing a refrigerator is heavy does. For a fish’s categories of smell, similarity may carry most of the load. For human concepts, how much of the load is carried by similarity is still an active debate.

Whether the missing pieces have to be built in, or can be learned from enough of the right experience, is also an open dispute. The digit-reading network generalized better because of what was built into it. The texture-bias networks learned shape once their experience changed. Even among researchers who argue that current networks are missing these pieces, there is no agreement about whether the pieces have to be built in.

Table 7.5 — *The limits of current classifiers, sorted by kind.*

Failure	What kind of failure it is	Where we take it up
Adversarial examples	A gap in pattern recognition	This module
Texture bias	A gap in pattern recognition, produced by the training data	This module
Overfitting	A gap in pattern recognition, reduced by inductive bias	This module
Failure out of distribution	A gap in pattern recognition	This module
Catastrophic interference	A gap in the learning procedure, under learning in sequence	This module; how brains avoid it, Module 10
Knowing what an object is like, beyond recognizing it	A mark of a later capacity	Modules 8 and 12
Reasoning about causes	A mark of a later capacity	Module 12
Systematicity	A mark of a later capacity	Modules 13 and 14

§7.2.14 — What Has to Be Built In?

Whether the missing pieces of current classifiers have to be built in, or can be learned, is a modern form of an old debate. In Module 2, we introduced it as the debate between nativists, who explain a capacity by what an animal starts out with, and empiricists, who explain it by learning and experience. We also drew that debate’s main lesson. The useful question is not whether a capacity is innate or learned. It is what exactly is claimed to be built in, because the candidates differ enormously. Pattern recognition lets us list some of the candidates precisely. Each of them is something an animal or a network could start out with.

The first way knowledge can be built into a pattern recognition system is the input space itself. Which receptors an animal has decides which feature space it starts in, and which differences it can register at all. No amount of learning lets a fish tell apart two smells that none of its receptors respond to differently. The input space can also make a problem easier, not only possible. A receptor or an early circuit that already responds to a combination of chemicals would hand the fish's later circuits an exclusive-or problem already solved. A built-in feature of that kind is an inductive bias supplied by the animal's lineage. It is the same bet a convolutional network makes in its wiring, placed instead in what the animal starts out with.

The second way knowledge can be built in is an architectural bias, an assumption built into how the units are wired. Convolution assumes that a feature means the same thing wherever it appears. A large random expansion layer, like the one in the fruit fly's brain, assumes that useful combinations will turn up among many random ones. Both assumptions are in place before any learning starts.

The third way knowledge can be built in is the learning rule, meaning what a weight change depends on. We sorted these into four kinds in Module 5. Backpropagation and competitive learning, given the same inputs, come to represent different things, because the two rules respond to different information. Animals also differ in what their rules link most readily. A rat links a new flavor with illness after a single pairing, even hours apart, but hardly links a flavor with a shock at all.

The fourth way knowledge can be built in is specific content. This could include a detector already wired for one particular predator, or caregiver. Newborn human babies, in their first hour, turn their eyes further to follow a pattern arranged like a face, than to one with the same parts scrambled.

Building *everything* in is not a good solution, though, for at least two reasons. The first is size. The number of possible combinations of features grows explosively. With 10 input features, there are 45 possible pairs, 120 possible triples, and more than a thousand possible combinations of all sizes. A fish has many dozens of kinds of smell receptor, so a detector for every combination is out of the question, and a lineage cannot know in advance which few combinations will matter.

The second reason it is a bad idea to build everything in is change. A built-in solution fits the world the animal's ancestors lived in. When that world changes, with a new predator or a new food, a built-in detector cannot follow, and learning can. In this sense, building in too much is a kind of overfitting, to the lineage's past. Hidden layers are valuable for exactly these reasons. Learning can find the combinations a task needs, in the world as it is now, without anyone listing them in advance. Natural selection can also arrive at useful combinations without listing them all. But this takes natural selection generations to accomplish what learning can do in a fraction of a lifetime. So for any given domain, whether an animal's solution is built in, learned, or a mixture of the two, and which would serve it better, are empirical questions.

Machine learning faces the same choice. Its practitioners call the built-in side **feature engineering**. This is when the engineer decides, before learning begins, what the inputs to a learner will be. Even the systems best known for learning from experience start from inputs chosen by people. The networks behind AlphaGo, which beat one of the world's strongest players at the board game Go in 2016, did not learn from simple photographs of Go boards. They received the positions of the stones directly, along with features that Go players consider important, such as how many open points surround each group of stones. A later version dropped those extra features and learned from the stone positions alone. Language models do not read images of printed text either. They learn from text already cut into words and pieces of words. In each case, someone decided which units probably carry the structure that matters, and learning starts from there. How far down learning has to reach is an open question for machines, just as it is for human and nonhuman animals.

Researchers who disagree about nativism are often disagreeing about different items among those four. Almost no one doubts that receptors are built in. Few doubt that some architectural biases are. The live disputes are over how much specific content has to be supplied in advance, and how much a general architecture and learning rule can extract from experience. For machines, the digit-reading network and the texture-bias networks are evidence on the two sides of that dispute.

For animals, the answer can differ even among fish. In some species, there is evidence that fish raised without ever encountering a predator react to its odor the first time they smell it. In others, the reaction is learned. A fish that smells a predator at the same time as chemicals released from the injured skin of another fish of its own kind comes to avoid the predator's odor afterward. That is associative learning of the kind we described in Module 5. A detector that is ready-made and a detector that is learned can produce the same behavior. Only experiments on animals raised without the experience can tell them apart. The cortical evidence in §7.3 bears on the same question from another direction. When

the input to a region of cortex is changed early in life, much of what the region comes to do follows its new input.

§7.3 — Implementational: Pattern Recognition in the Cortex

§7.3.1 — The Oldest Cortex

In mammals, the first cortex a smell reaches is the **piriform cortex**. It is a sheet with three layers, older in form than the six-layered neocortex. It is also unusual in where its input comes from. Signals from the other senses reach the cortex through a relay in the thalamus. Smell does not. The olfactory bulb, the first stop for signals from the nose, sends its output straight to the piriform cortex. How the bulb's signals are produced is a question about the senses, and we take it up in Module 8.

The piriform cortex is built around the **pyramidal neuron**, which is also the main excitatory cell of the neocortex. Its cell body is roughly triangular. A long dendrite, called the apical dendrite, runs from the top of the cell toward the surface of the cortex, and shorter dendrites spread out near the cell body. The bulb's input lands on the far ends of the apical dendrites. Each pyramidal neuron's axon leaves the piriform cortex, but on the way it sends side branches to other pyramidal neurons nearby, so the cells excite one another. These are recurrent connections, of the kind that let a circuit in Module 4 hold a state. Inhibitory cells sit among the pyramidal neurons and damp their activity down (Figure 7.11).

The plan is old, but not every part of it is equally old. The lamprey's pallium also has three layers, and smell also arrives on the far ends of its cells' dendrites. But the lamprey's cells lack the shorter dendrites near the cell body, so the full pyramidal neuron cannot be dated back that far.

Consider a mouse sniffing at the mouth of a burrow. It needs two things from its piriform cortex. It needs to tell apart two smells that are almost the same, such as its own nest and a neighbor's. And it needs to know a familiar smell from a faint trace of it. Those are the two demands of §7.1.2, discrimination and generalization, and the piriform cortex has been studied for both.

§7.3.2 — Pulling Similar Smells Apart

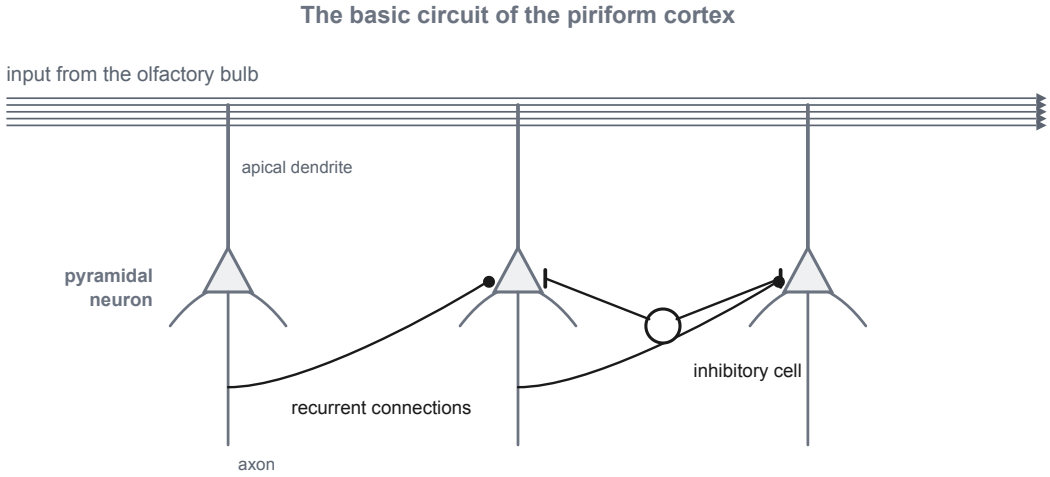


Figure 7.11 — *The basic circuit of the piriform cortex. Input from the olfactory bulb arrives on the far ends of the pyramidal neurons' apical dendrites. Side branches of each pyramidal neuron's axon excite its neighbors, and inhibitory cells damp the activity. After Linster and colleagues (2003).*

What the piriform cortex does with a smell has been measured directly. In one study, researchers imaged the activity of many cells at once in the piriform cortex of anesthetized mice, while presenting

one odor after another. Each odor activated a small set of the cells, between about 3 and 15 percent of those imaged. The active cells were scattered. Cells that responded to the same odor were not clustered together, and there was no map. A cell's neighbor was as likely to respond to a quite different odor as to the same one.

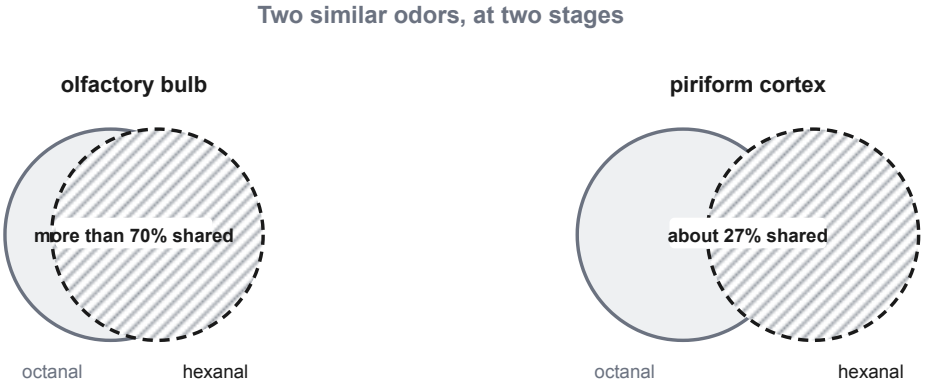


Figure 7.12 — *Two similar odors, compared at two stages. In the olfactory bulb, the inputs activated by two odors with almost the same chemical structure overlap by more than 70 percent. In the piriform cortex, the cells the same two odors activate overlap by only about a quarter. After Stettler and Axel (2009).*

The most telling result came from pairs of similar odors. Two odors with almost the same chemical structure activate overlapping sets of inputs in the olfactory bulb, with more than 70 percent of the active inputs shared. In the piriform cortex, the same two odors shared only about a quarter of their active cells. Odors with unrelated structures shared between 5 and 15 percent (Figure 7.12). So two smells that look alike at the bulb look much less alike in the piriform cortex. Making overlapping inputs less alike, so that a later stage can tell them apart, is called **pattern separation**. It is the discrimination of §7.1.2, carried out by a circuit.

Single cells fit the same picture. One cell could respond to two odors with unrelated structures. So no single cell stands for an odor. What identifies an odor is which set of cells is active.

The researchers proposed an explanation. If each piriform cell receives input from a random handful of the bulb's outputs, these properties would follow. And if the wiring is random, then what an odor means cannot be built into the piriform cortex. It has to come from experience. That is the second answer of §7.2.7, random wiring with a trained output, in a mammal. Two cautions apply. How much two odors overlap depends on how strongly a cell must respond to be counted as active. And random wiring is a model that fits the data, not something the study observed. The study also did not count cells, so it says nothing about whether the piriform cortex spreads its input across more cells than it receives from.

§7.3.3 — Completing a Smell from Part of It

The recurrent connections suggest a second job. Suppose that, when a smell activates a set of pyramidal neurons, the connections among those active cells are strengthened, by Hebb's rule from Module 5. The next time only part of the set is activated, the strengthened connections can activate the rest. Storing patterns in the connections among a circuit's own cells is called **auto-association**, and recovering a whole stored pattern from part of it is **pattern completion**. It is the generalization of §7.1.2, carried out by a circuit. A faint or partial version of a familiar smell produces the response the whole smell would.

The piriform cortex was proposed as a memory of this kind, because its input spreads widely and its cells excite one another. Network models built on its circuit do store patterns and complete them from partial input. But the support comes mostly from models, and direct tests in animals are few. For the mouse, the proposal would explain how a trace of its nest's smell, carried on a draft, can be enough. The

hippocampus is the other place where pattern separation and pattern completion have both been proposed, and we take up its circuits in Modules 9 and 10.

§7.3.4 — A Fly, a Bird, and a Fish

The clearest evidence for random wiring comes from an insect. In the fruit fly, smell signals are sorted into about 50 kinds of input, which feed a structure called the **mushroom body**. There they reach about 2,000 cells, called Kenyon cells. Each Kenyon cell receives input from a few of the 50 kinds. When the connections of 200 Kenyon cells were traced, the combinations looked random, and they were not the same from fly to fly. Kenyon cells respond sparsely, so each odor activates only a few of them (Figure 7.13).

The mushroom body is the second answer of §7.2.7 in its plainest form: an expansion from about 50 inputs to about 2,000 cells, wired at random and never trained. Learning happens downstream, at the connections from the Kenyon cells to the mushroom body's output neurons. There, a signal carried by dopamine settles which connections change, which is the three-factor rule of Module 5.

The fly and the mouse are separated by more than half a billion years of evolution, and their last common ancestor had nothing like either structure. So the resemblance between the mushroom body and the piriform cortex is best read as convergent evolution, in the terms of Module 2. Two lineages arrived at a similar arrangement for a similar problem. That reading is an inference from the family tree of animals, not something either study tested.

Birds and turtles have no neocortex either, but they have cells proposed, from the genes they express, as counterparts of the neocortex's layer-4 cells. In Module 6, we described the dispute over how the bird pallium relates to the mammalian one, and that dispute continues. A fish can also recognize objects without a neocortex, though the evidence is less clean than it sounds. Goldfish learned to choose a photograph of one toy animal over another, and then chose correctly when the photographs were rotated to new orientations. But the two toys always differed in color, and the fish may have used color alone.

§7.3.5 — One Plan, Many Areas

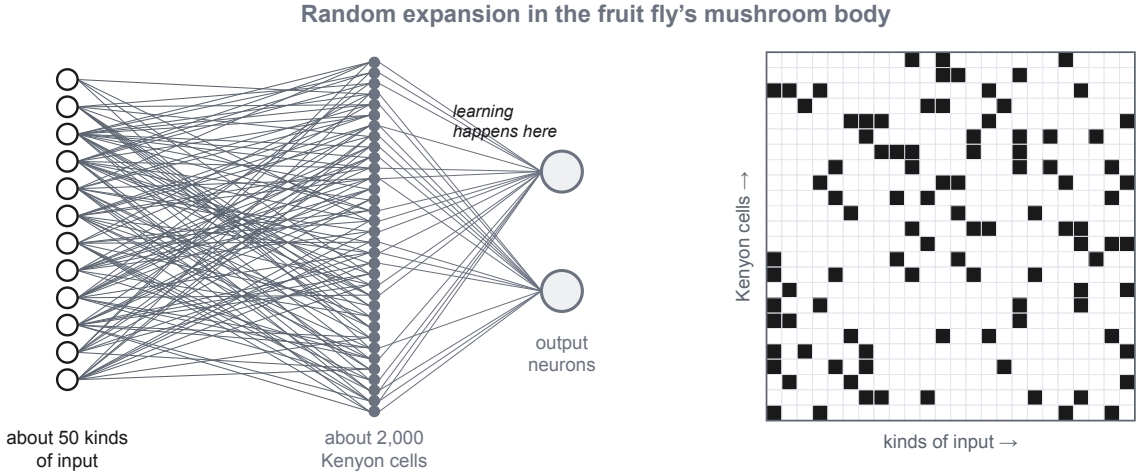


Figure 7.13 — *Random expansion in the fruit fly's mushroom body. Left: about 50 kinds of smell input reach about 2,000 Kenyon cells, each of which receives a few of them. Right: the connections as a grid, one row per Kenyon cell and one column per kind of input. The grid is drawn at random, to show the kind of pattern tracing found: no rows alike, and no structure. After Caron and colleagues (2013).*

In mammals, most of the pallium is neocortex. In Module 6, we described the neocortex as a nearly uniform sheet, and that description survives with one correction. The neocortex is not one circuit copied

identically across the sheet. It is one circuit plan, repeated with variations. Every area has the same classes of cells, connected in much the same way, and areas differ mainly in how many cells of each class they have and how strongly they connect. Areas resemble one another the way hands resemble feet. The biggest differences are in layer 4, the layer that receives input from the thalamus. It is prominent in the areas that receive signals from the senses, and nearly absent in the motor cortex. Most of a cortical cell's input comes from other cortical cells, not from outside. Even in the input layer of the visual cortex, connections arriving from the thalamus make up less than a tenth of the excitatory synapses. Whether one circuit plan means one computation everywhere is still an open question.

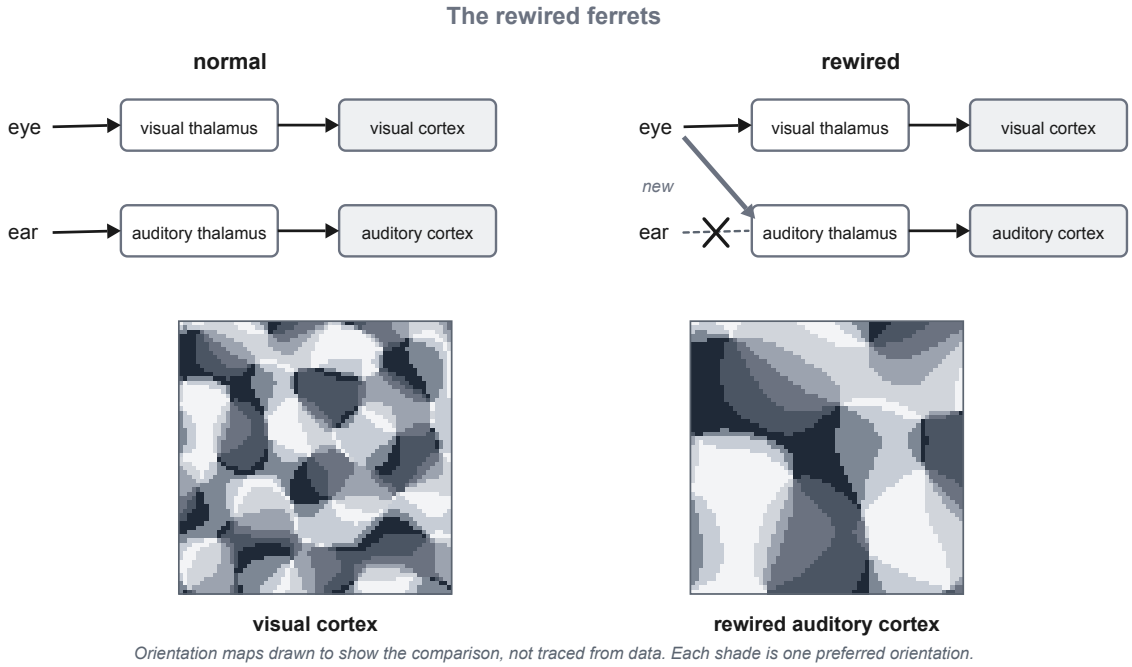


Figure 7.14 — *The rewired ferrets. Top: in a normal ferret, signals from the eye pass through the visual part of the thalamus to the visual cortex, and signals from the ear pass through the auditory part to the auditory cortex. In a rewired ferret, axons from the eye grow into the auditory part of the thalamus, which passes their signals to the auditory cortex. Bottom: orientation maps drawn to show the comparison, with each shade standing for one preferred orientation. The map in rewired auditory cortex has larger patches and about a quarter as many points where all the orientations meet. After Sharma, Angelucci and Sur (2000).*

The strongest evidence on that question comes from rewired ferrets. In ferrets a day old, researchers cut off the auditory part of the thalamus from its normal input. Axons from the eyes then grew into it, and it passed their visual signals on to the auditory cortex, along its normal connections. As the animals grew up, the auditory cortex developed cells that responded to edges at particular orientations, like cells in the visual cortex. The cells were arranged in an orientation map, but a less orderly one than the visual cortex has, with about a quarter as many of the points where all the orientations meet (Figure 7.14).

In a second study, ferrets rewired on one side were trained to go to one spout for a sound and to another for a light. Then the normal visual pathway on the rewired side was cut, so that a light in one part of the visual field could reach the cortex only through the rewired route. All three animals that were fully tested treated those lights as lights, not as sounds. When their auditory cortex was then removed, their responses fell to about chance.

These results show that input shapes what a cortical area does. They do not show that areas are interchangeable, or that the ferrets saw normally. The rewired map was less orderly, the animals saw less sharply through the rewired route, and the researchers describe the area as becoming more visual-

like, not visual. Still, an area that would normally have handled sound came to treat light as light, because light was what reached it.

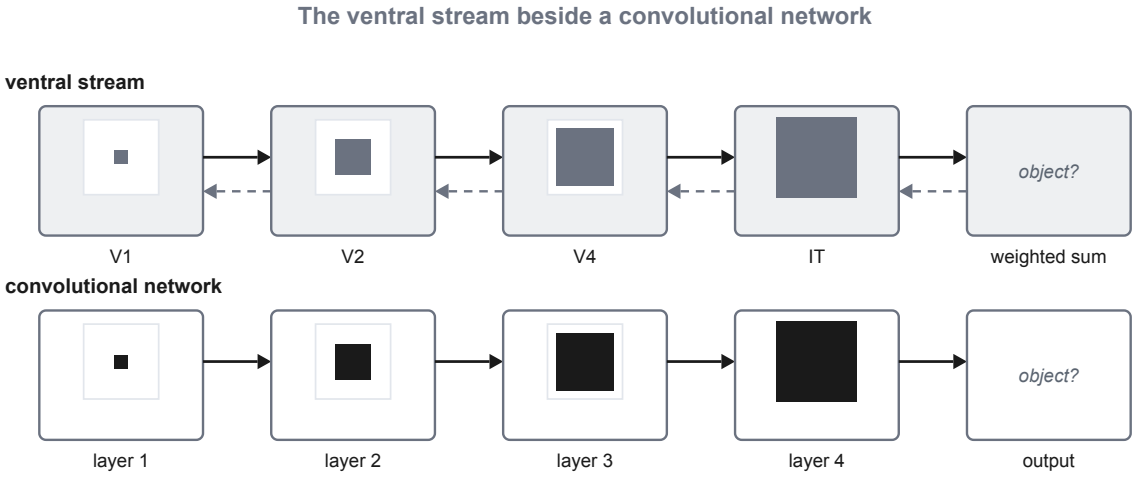


Figure 7.15 — *The ventral stream beside a convolutional network. Up both, receptive fields grow larger at each stage, from small fields tuned to oriented edges to large fields that respond to whole objects. Feedback connections, drawn as dashed arrows pointing back down, run between the cortical areas and are absent from the network.*

§7.3.6 — The Visual Cortex Beside a Network

In Module 1, we followed visual signals from the eyes through a relay in the thalamus to the primary visual cortex, V1. There, the neuroscientists David Hubel and Torsten Wiesel found cells that respond to edges at particular orientations. Their 1962 study in cats sorted those cells into two kinds. Each kind is described by its **receptive field**, the region of the visual field where a stimulus changes the cell's activity, together with the kind of stimulus that does it. A **simple cell** has a receptive field divided into regions where light excites the cell and regions where light inhibits it, so its response to any pattern can be predicted from a map of those regions. A **complex cell** responds to an edge or bar of the right orientation anywhere within a larger receptive field.

Hubel and Wiesel suggested how a complex cell might be built. It could combine the outputs of several simple cells that prefer the same orientation at slightly different positions. They called the proposal "obviously tentative". But it is the arrangement Fukushima built into the neocognitron, and the arrangement we described in §7.2.9 as convolution followed by pooling. The simple cells detect a feature at each position, and the complex cell ignores exactly where.

Beyond V1, a series of areas called the **ventral stream** runs along the underside of the brain, through areas known as V2 and V4, to the inferior temporal cortex, or IT. Up the stream, receptive fields grow larger, and cells keep their preferences over wider changes in an object's position and size. But no single cell becomes fully invariant. What a cell has is **tolerance**, keeping its response over a limited range of changes. And the cells most selective for a particular shape are the least tolerant of changes to it. A few hundred neurons in IT are enough for a weighted sum of their activity to report which object is in view. That weighted sum is a single unit's straight line, of the kind in §7.2.3, at the top of a hierarchy that has re-represented the input until one line will do (Figure 7.15).

A readout of that kind has a criterion as well as a sensitivity, and the distinction of §7.1.4 has a physical form. In a unit, the criterion is set by the threshold or the baseline, as we saw in §7.2.3. In a neuron, the counterpart is how excitable the cell is, meaning how close its resting state sits to firing. Excitability is what neuromodulators change. In Module 4, a hungry worm and a fed worm ran the same circuit with different chemistry. In the terms of this chapter, the worm's condition moved its criterion, while the input and the wiring stayed the same. Sensitivity has a different physical home. It depends on how

well the input a neuron receives separates the things that matter. So it lives in the representation built upstream: in the pattern separation of the piriform cortex, the expansion of the mushroom body, and the stages of the ventral stream.

The comparison with convolutional networks holds in part. Among networks of this kind, the ones that are better at categorizing objects are also better at predicting how neurons in IT respond. In 2014, the best such network predicted about half of the variation in IT responses that could be predicted at all. That network was not trained by backpropagation. The comparison also has limits. The cortex sends feedback from each area to the area before it, as well as signals forward, and the networks in these comparisons have no feedback. Whether fast recognition of a familiar object needs that feedback is debated. And some of the resemblance may rest on a shared sensitivity to texture, the bias of §7.2.11.

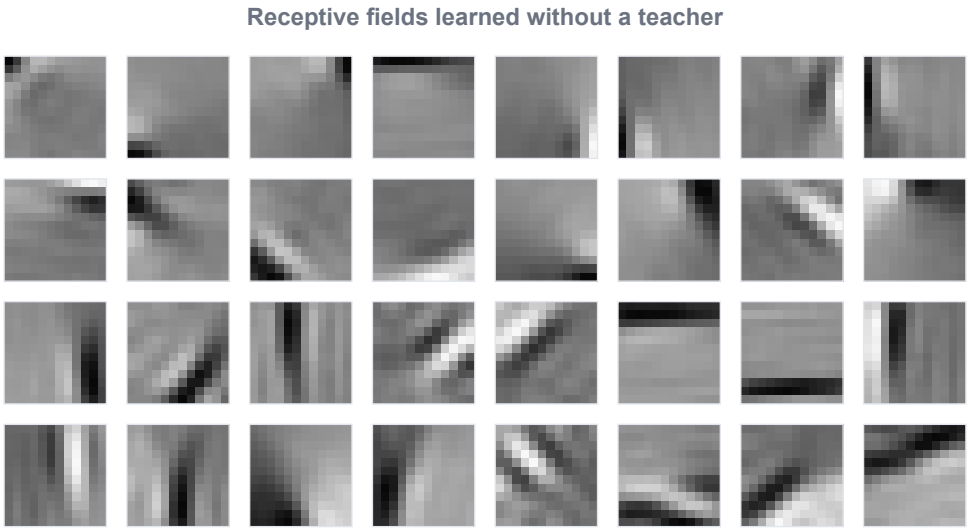


Figure 7.16 — *Receptive fields learned by a rule with no teacher, from small patches of two photographs, when the rule must describe each patch with as few active units as possible. Each square shows what one unit responds to, light against dark. Most are small, elongated and oriented, like the receptive fields of simple cells. Computed for this chapter by a related method, independent component analysis, after Olshausen and Field (1996).*

§7.3.7 — Could the Cortex Pass Errors Back?

In §7.2.7, we left open whether a brain could learn by passing errors back. Part of the answer is already in hand. In Module 6, we described the pallium as learning with no teaching signal at all, and noted that there is no dedicated teaching pathway to find. Prediction learning fits the second point and qualifies the first. The next input arrives at the pallium anyway, so no separate pathway is needed to carry a target to it. What the pallium lacks is an external teacher, not necessarily an error.

How the error would be delivered is harder. Backpropagation, taken literally, needs three things that no one has found in the cortex. The feedback connections would have to carry the same weights as the forward connections they run beside. The error signals would have to be positive or negative, over a huge range of values. And the feedback would have to change synapses without changing the activity of the neurons it passes through. The method's inventors themselves wrote, in 1986, that it was not a plausible model of learning in brains.

As the evidence stands, there is no direct evidence for anything like backpropagation in the cortex, and none that rules it out. One family of proposals keeps the idea and changes the mechanism. Feedback changes a neuron's activity, and the neuron learns from the difference between its activity before the feedback arrived and after. Learning of that kind can approximate what backpropagation computes, using only signals a neuron could have. A related proposal, *predictive coding*, holds that feedback carries predictions back down the hierarchy, and that the forward connections carry what the

predictions missed. Its central prediction, neurons dedicated to carrying the error, lacks clear support, and its originator says that much of it remains to be tested. Both proposals are open hypotheses.

§7.3.8 — Few Active at Once

In Module 3, we found that a spike is expensive, and that the brain's energy budget allows only a small share of cortical neurons to be strongly active at any moment. That is sparse coding, and the piriform cortex and the mushroom body both show it. What is new here is what a sparse code does for recognition.

Two results connect sparse codes to this chapter. In the first, researchers gave a learning rule with no teacher a large number of small patches cut from photographs of natural scenes. The rule had two goals: describe each patch accurately, and use as few active units as possible to do it. The units developed receptive fields like those of simple cells in V1, small, elongated, and tuned to edges at particular orientations (Figure 7.16). The two goals alone were enough to produce the main properties of simple cells. That does not show that V1 learns this way, only that a rule of this kind could produce what V1 has. The second result is the one from §7.2.7. When only a few of many units are active for each input, many more ways of dividing the inputs can be drawn with a single straight line, so a single trained layer can do what would otherwise take several. The energy budget limits how many neurons can be active at once, and a code that sparse also suits recognition. How sparse cortical codes actually are is still argued.

Close — One Plan, Many Uses

At the end of Module 6, we asked whether a structure that does the same thing to every kind of input is doing something very general, or nothing in particular. The answer we have built in this chapter is that it is doing something very general. It re-represents whatever reaches it. Inputs that call for different responses are pulled apart, and inputs that come from the same thing are brought together, until a single straight line can do the rest. The mouse's piriform cortex pulls two similar smells apart, so that the cells reading its output can tell them apart. An auditory cortex fed by the eyes came to treat light as light. The stages of the ventral stream turn an object at many positions and sizes into a pattern that a few hundred cells can report. The fish's pallium faces the same problem with other circuits, and the fly's mushroom body with a different nervous system altogether.

The problem was the same for each of them. A fish, a fly, a mouse, and a network each have to tell apart what calls for different responses, and treat alike what comes from the same thing, from inputs that are never twice the same. Each handles it with different machinery. That is the lesson of Module 6, seen in a single capacity. The computational level can hold steady while the implementation is rebuilt beneath it.

Recognizing a pattern tells an animal what a thing is. It does not, by itself, say why the thing is there, what would happen if the animal acted on it, or how the parts of a thought combine. Those questions take more than recognition, and they return in later modules.

Throughout this chapter, we took the inputs as given: a smell already turned into activity in the olfactory bulb, an image already relayed through the thalamus. Recognizing objects and recognizing words are specializations of the problem in this chapter, and each adds demands of its own. An object has a three-dimensional shape that has to be recovered from flat images, and a word is a pattern whose meaning is set by a language. We take up objects, and the senses that deliver their images, in Module 8, and words in Module 13.

Further Reading

Six places to go next, each one taking up a thread this chapter could only pull on briefly.

Gregory L. Murphy, *The Big Book of Concepts*, MIT Press, 2002. The fullest account of the psychology behind §7.2.1–§7.2.5, and the best place to see why the contest between prototype and exemplar models has been so hard to settle. Chapters 3 and 4 work through the experiments and their

weaknesses, and later chapters take up the view, mentioned in §7.2.13, that categories are held together by knowledge about why their members are alike.

Eleanor Rosch and Carolyn B. Mervis, "Family resemblances: studies in the internal structure of categories", *Cognitive Psychology* 7(4), 1975, pages 573–605. The paper behind §7.1.5's typicality and its family resemblance. It is still readable, and it shows how much can be learned about categories from asking people simple questions carefully.

Daniel D. Stettler and Richard Axel, "Representations of odor in the piriform cortex", *Neuron* 63(6), 2009, pages 854–864. The measurements behind §7.3.2. The paper's discussion sets out the case that the piriform cortex is wired at random and must learn what its patterns mean, and compares it with the insect mushroom body.

James J. DiCarlo, Davide Zoccolan, and Nicole C. Rust, "How does the brain solve visual object recognition?", *Neuron* 73(3), 2012, pages 415–434. The ventral stream of §7.3.6, explained as a series of stages that re-represent an image until a weighted sum can read off the object. It is the clearest statement of the idea of tolerance, and of what remains unknown about how the stages learn.

Timothy P. Lillicrap, Adam Santoro, Luke Marris, Colin J. Akerman, and Geoffrey Hinton, "Backpropagation and the brain", *Nature Reviews Neuroscience* 21(6), 2020, pages 335–346. The question of §7.3.7, argued by researchers who think the cortex may well approximate backpropagation. They set out exactly what literal backpropagation would require, and the proposals that might deliver its effects without it.

Brenden M. Lake, Tomer D. Ullman, Joshua B. Tenenbaum, and Samuel J. Gershman, "Building machines that learn and think like people", *Behavioral and Brain Sciences* 40, 2017, e253. The case, made in §7.2.13, that current networks recognize patterns well and are missing other things. It is a *BBS* target article, printed with dozens of commentaries that argue the question from every side.

Glossary

adversarial example — An input changed slightly and deliberately, so that a network gives a confident wrong answer although a person would see no difference.

auto-association — Storing patterns in the connections among a circuit's own cells, so that part of a stored pattern can reactivate the rest.

backpropagation — An extension of the delta rule to networks with hidden layers. Each hidden unit's error is computed from the errors of the units it feeds, passed backward through the same connections the activity passed forward through.

catastrophic interference — A drastic loss of old learning when a network learns new patterns after old ones, because the new learning moves weights that the old patterns depended on.

category — A set of things treated as the same kind, for some purpose.

classical view — The view that a category is defined by features that are individually necessary and jointly sufficient for membership, so that everything with all the defining features belongs and nothing without them does.

competitive learning — Learning in which units that receive the same input inhibit one another until one wins, and only the winner moves its weights toward the input. It needs no teacher.

complex cell — A cell in the primary visual cortex that responds to an edge or bar of a particular orientation anywhere within a fairly large receptive field.

convolution — Applying the same small feature detector, with the same weights, at every position in an input, so that a feature is found wherever it falls.

correct rejection — In signal detection theory, judging a target absent when it is absent.

criterion — In signal detection theory, the amount of evidence at which a decision changes from "no" to "yes". Moving it trades misses for false alarms, and leaves sensitivity unchanged.

decision boundary — The line through the space of possible inputs that separates the combinations a unit fires to, from the combinations it does not.

decision-bound model — A model of categorization that divides the feature space into regions, one for each response, and responds according to the region an input falls in.

dimensionality expansion — Spreading an input across many more units than it started with, which makes it more likely that a single straight boundary can separate the cases that matter.

discrimination — Treating differently the inputs that call for different responses, even when those inputs overlap.

distributed representation — A code in which each thing is a pattern of activity across many units, and each unit takes part in representing many things.

exemplar — A stored example of a category. An exemplar model categorizes a new input by its summed similarity to the stored members of each category.

false alarm — In signal detection theory, judging a target present when it is absent.

feature engineering — Deciding, before learning begins, what the inputs to a learning system will be.

feature space — A space with one axis for each feature of an input, in which each input is a single point.

generalization — Treating alike the inputs that call for the same response, even when those inputs barely overlap.

generalization gradient — The fall-off in responding to a new input as it differs more and more from the input an animal was trained on.

hidden layer — A layer of units between a network's input and its output, driven directly by neither the world nor the response. Its units come to stand for combinations of the inputs.

hit — In signal detection theory, judging a target present when it is present.

inductive bias — A tendency to find one thing easier to learn than another, established before the learning begins, whether by the animal's lineage or by what it has already learned.

invariance — Recognizing a thing across changes that alter its input a great deal, such as changes of distance, position, or viewpoint.

k-nearest-neighbor classification — Assigning a new input to whichever category is most common among the *k* stored examples most similar to it, where *k* is a number chosen in advance.

linear separability — The property of two categories that can be divided by a single straight line, or a single flat surface, through the feature space, with all of one category on one side and all of the other on the other.

localist representation — A code in which each thing has its own unit, so that which unit is active is itself the information.

miss — In signal detection theory, judging a target absent when it is present.

mushroom body — A structure in the insect brain where smell inputs are spread across many more cells, the Kenyon cells, through connections that appear random, with learning at the connections to its output neurons.

out-of-distribution generalization — Generalization to inputs unlike anything a learner was trained on.

overfitting — Fitting the training examples so closely, including their accidents, that performance on new examples suffers.

pattern completion — Recovering a whole stored pattern from part of it.

pattern recognition — Identifying something from a combination of activity across many inputs, no one of which identifies it alone.

pattern separation — Making overlapping inputs less alike, so that a later stage can tell them apart.

piriform cortex — The three-layered olfactory cortex of mammals, which receives the olfactory bulb's output directly, without a relay in the thalamus.

pooling — Combining the detections of a feature over a small neighborhood, so that a small shift in the input no longer changes the result.

prototype — A summary of a category, such as its average member. A prototype model categorizes a new input by the category whose prototype it is closest to.

pyramidal neuron — The main excitatory cell of the cortex, with a roughly triangular cell body, a long apical dendrite reaching toward the surface, and shorter dendrites near the cell body.

receptive field — The region of the sensory world where a stimulus changes a cell's activity, together with the kind of stimulus that does it.

representation — The internal form in which a mind holds information about the world. For example, a spoken greeting held first as speech sounds, then as words, then as a speaker and a meaning. One of the field's two central questions is *how is information represented?*

sensitivity — In signal detection theory, how far apart the evidence for "present" and the evidence for "absent" lie, whatever the criterion.

signal detection theory — A framework for decisions about whether something is present, which separates how well the evidence distinguishes the two cases from where the line between "yes" and "no" is drawn.

similarity — Closeness between two inputs in a feature space. How it is measured, and in which space, differs from one model to another.

simple cell — A cell in the primary visual cortex whose receptive field is divided into regions where light excites it and regions where light inhibits it, so that its response can be predicted from a map of those regions.

sparse coding — Activity in which only a small proportion of neurons are strongly active at any one moment. It is a consequence of the energy budget rather than an inefficiency.

structural credit assignment — Working out which connections inside a network were responsible for an error at its output.

systematicity — The ability to combine known parts in new ways, so that understanding one combination brings with it the understanding of others built from the same parts.

test set — Examples kept apart from training and used only to evaluate a learner.

texture bias — A tendency to categorize images by their surface texture rather than their shape, found in networks trained on standard collections of photographs.

tolerance — A cell's keeping its preferred response over a limited range of changes to a stimulus, such as changes of position or size.

topographic map — An arrangement in which neighboring points in the world are handled by neighboring cells, so that the layout of the tissue mirrors the layout of what it represents.

training set — The examples a learner learns from.

typicality — How good an example of a category an item is, as when a robin is a better example of a bird than a penguin.

ventral stream — The series of visual areas running from the primary visual cortex along the underside of the brain to the inferior temporal cortex, where cells respond to whole objects.